



THE UNIVERSITY OF
MELBOURNE

COMP90050 Advanced Database Systems

Semester 1, 2026

Lecturer: Farhana Choudhury (PhD)

Live lecture – Week 4





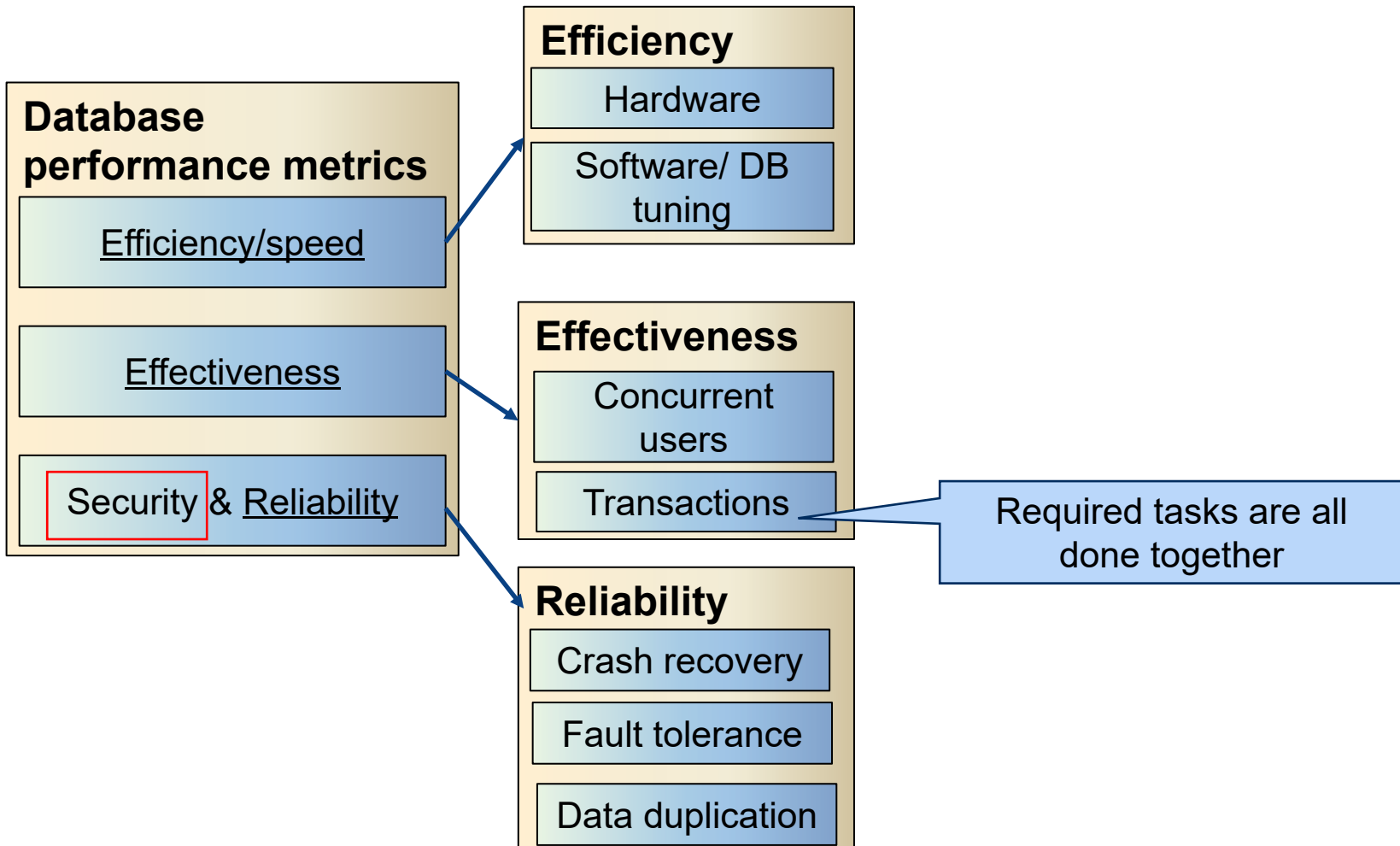
Quizzes

Quiz 5 next week!

Contents of Week 2 and Week 3 lectures and corresponding tutorials.

[Not Week 5 tutorial contents]

Today's lecture





This week's contents

Transactions

the building block of databases as consistency, concurrency, crash recovery – all these powerful and important database properties are developed based on transaction concepts

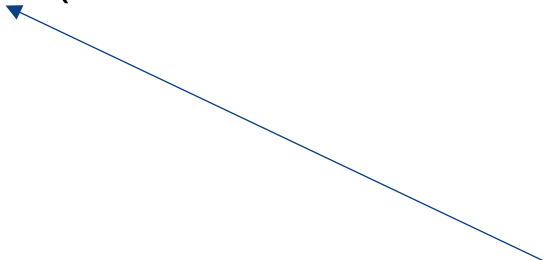
- Some related concepts (connecting SQL with a programming language, variable types, etc.)
- Types of transactions
 - Flat transactions
 - Flat transactions with save points
 - Nested transactions
- Transaction processing monitor



Flat Transaction

Everything inside BEGIN WORK and COMMIT WORK is at the same level; that is, the transaction will either survive together with everything else (commit), or it will be rolled back with everything else (abort)

```
exec sql BEGIN WORK;  
    AccBalance = DodebitCredit(BranchId, TellerId, AcclId, delta);  
    send output msg;  
exec sql COMMIT WORK;
```



Can be a very long running transaction
with many operations



```
/* Withdraw money -- bank debits; Deposit money – bank credits */
```

```
Long DoDebitCredit(long BranchId,  
    long TellerId, long AccId, long AccBalance, long delta){  
    exec sql UPDATE accounts  
    SET AccBalance = AccBalance + :delta  
    WHERE AccId = :AccId;  
  
    exec sql SELECT AccBalance INTO :AccBalance  
    FROM accounts WHERE AccId = :AccId;  
  
    exec sql UPDATE tellers  
    SET TellerBalance = TellerBalance + :delta  
    WHERE TellerId = :TellerId;  
  
    exec sql UPDATE branches  
    SET BranchBalance = BranchBalance + :delta  
    WHERE BranchId = :BranchId;  
  
    Exec sql INSERT INTO history(TellerId, BranchId, AccId, delta, time)  
    VALUES( :TellerId, :BranchId, :AccId, :delta, CURRENT);  
    return(AccBalance);  
}
```

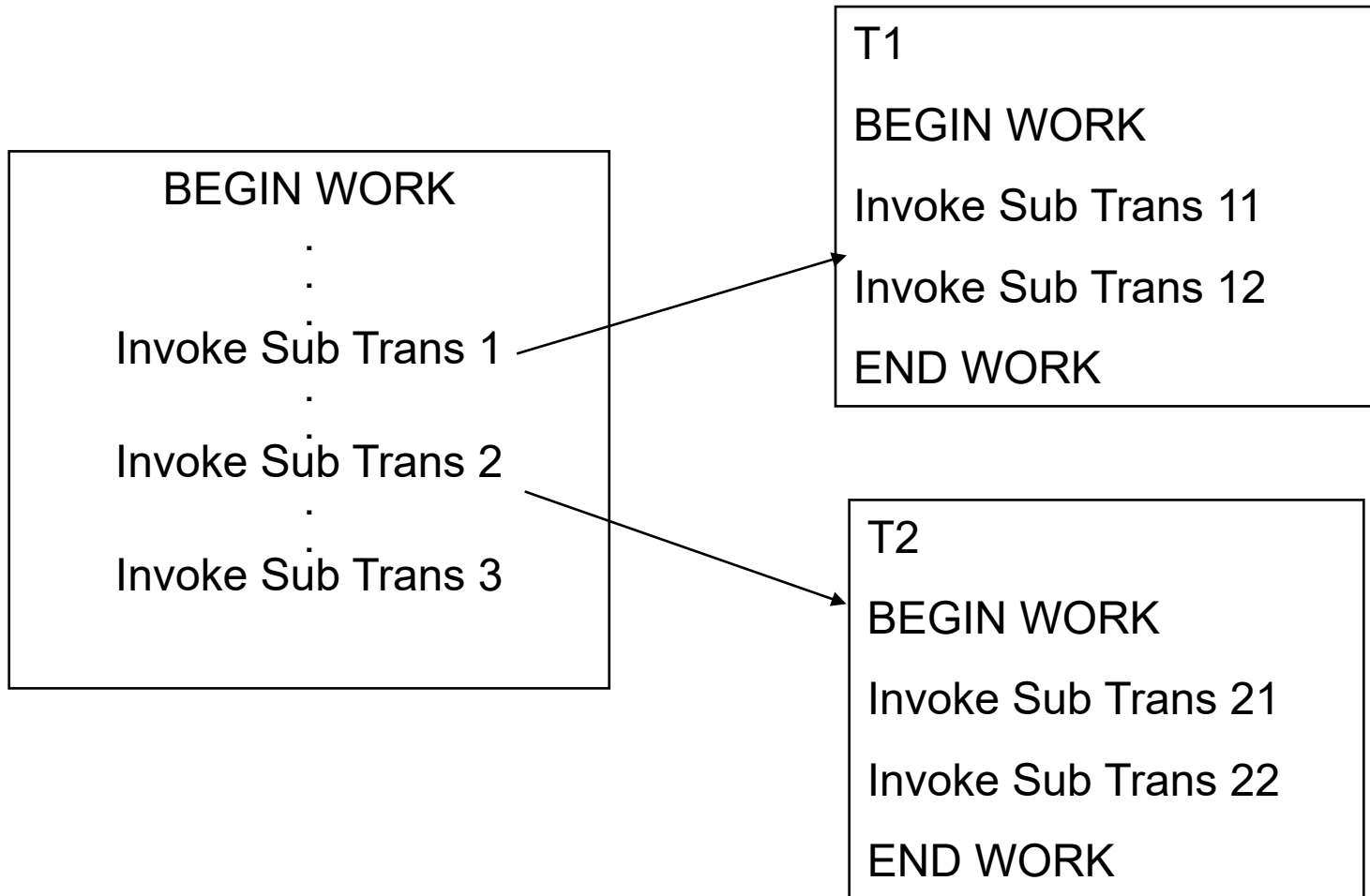
Limitations of Flat Transactions?

Solution – flat transaction with savepoints

Resources:

<https://dev.mysql.com/doc/refman/9.0/en/savepoint.html>

Nested Transactions





Nested Transactions

Commit rule

- A subtransaction can either commit or abort, however, **commit cannot take place unless the parent itself commits.**
- Subtransactions have A, C, and I properties but not D property unless all its ancestors commit.
- Commit of a sub transaction makes its results available only to its parents.

Roll back Rules

If a subtransaction rolls back, all its children are forced to roll back.

What are the advantages?

Time for a poll - [Pollev.com/farhanachoud585](https://pollev.com/farhanachoud585)





Embedded SQL example in C

(Open Database Connectivity)

```
int main()
{
    exec sql INCLUDE SQLCA; /*SQL Communication Area*/
    exec sql BEGIN DECLARE SECTION;
        /* The following variables are used for communicating
           between SQL and C */

    int OrderID; /* Employee ID (from user) */
    int CustID; /* Retrieved customer ID */
    char SalesPerson[10] /* Retrieved salesperson name */
    char Status[6] /* Retrieved order status */

    exec sql END DECLARE SECTION;

    /* Set up error processing */
    exec sql WHENEVER SQLERROR GOTO query_error;
    exec sql WHENEVER NOT FOUND GOTO bad_number;
```

Proper error handling is important!

What can go wrong in this example?

(Open Database Connectivity)

```
int main()
{
    .....//code to get data from database
    Printf("%d", ((get_salary(EMPID)/80000);

    /* Set up error processing */
    WHenever DIVIDE_BY_ZERO GOTO inf_error;
    WHenever NOT_PERMITTED GOTO permission_err;
    WHenever NOT_FOUND GOTO bad_number;
```

Time for a poll - [Pollev.com/farhanachoud585](https://pollev.com/farhanachoud585)



Proper error handling is important! - Inferential attack/data leak

Transaction Processing Monitor

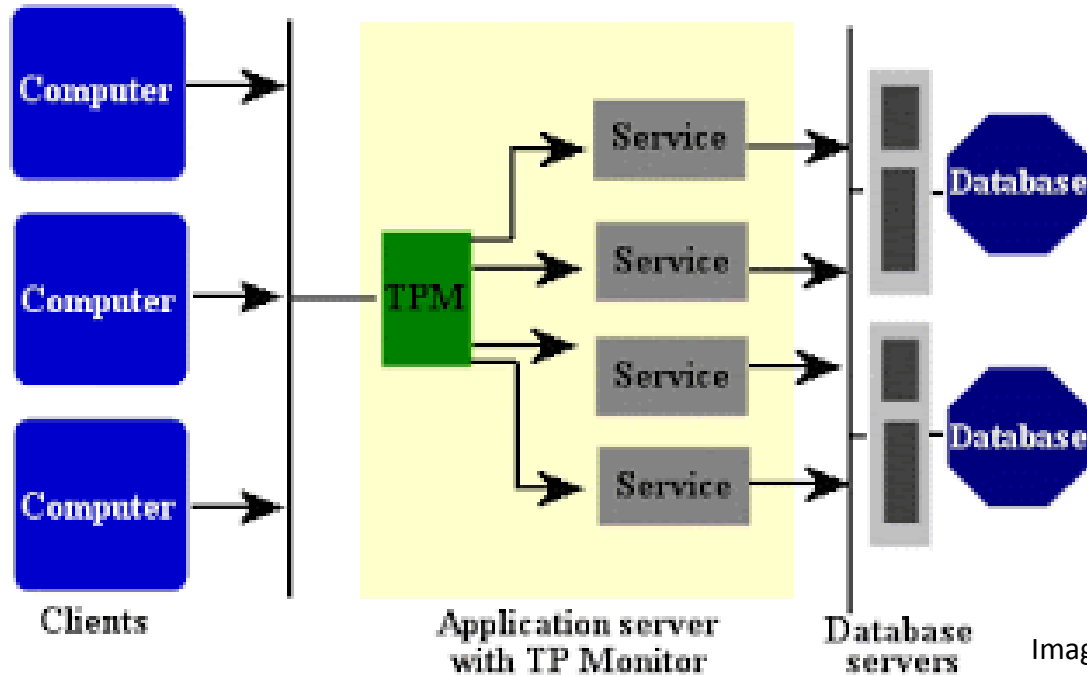


Image source: <http://3.bp.blogspot.com>

Integrates other system components and manage resources.

- Manages the transfer of data between clients and servers
- breaks down applications or code into transactions and ensures that all databases are updated properly
- It also takes appropriate actions if any error occurs

Core Concepts of Database management system

